1강. 프런트엔드 웹 개발(HTML, CSS)

React

강사 소개

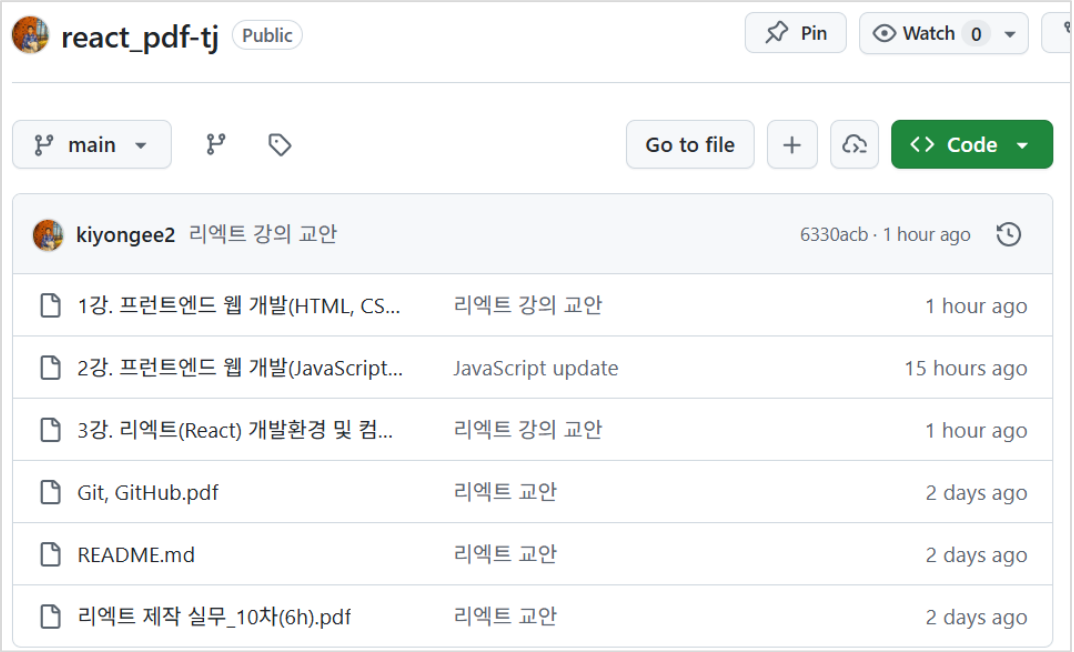
이름 : 김 기 용

학력 및 경력:

- (현) 더조은컴퓨터 아카데미 리엑트 실무 과정 강의
- (전) SPACECL K-MOVE 일본 취업 풀스택 웹 개발자 강의
- (전) 하이미디어, KH 정보교육원 Java WEB 개발자 강의
- (전) 에듀윌, 그린컴퓨터 Python WEB 개발자 강의
- 한국방송통신대학교 이학사(컴퓨터과학)
- 서울시립대학교 행정학사

교안 자료

- 깃허브 주소: https://github.com/kiyongee2/react_pdf-tj
- 강사 이메일 : sudo2100@naver.com
- 강사 홈페이지 : http://giyong.info/

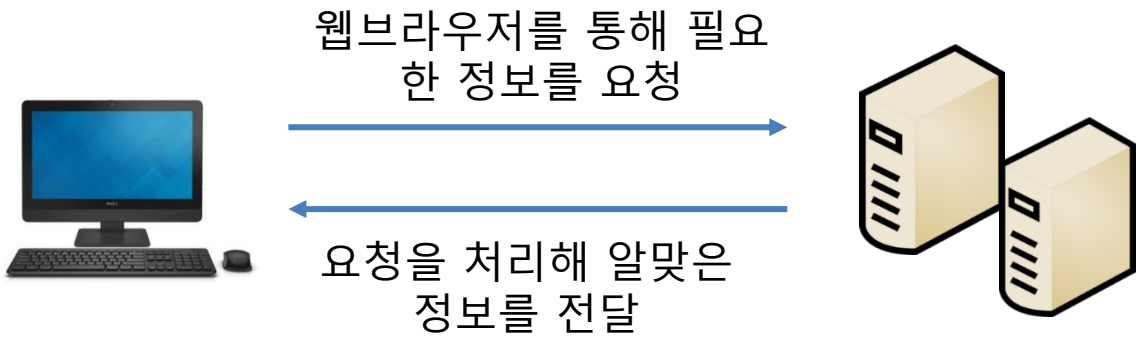


프런트엔드 vs 백엔드

- 웹 프로그래밍의 구분
  - **프런트엔드(Front-end) 개발**

    –> 서버에서 받아 온 정보를 웹 브라우저에 어떻게 보여줄 것인지를 프로그래밍
  - **백엔드(Back-end) 개발**

    –> 서버에서 사용자의 요청을 처리하거나 데이터를 관리하는 프로그래밍



클라이언트와 서버

◆ Client vs Server

클라이언트(Client)측 언어

- HTML
- CSS
- JavaScript

서버(Server)측 언어

- Java(Spring)
- 파이썬(Django)
- PHP
- JavaScript(Nestjs)

프런트엔드 웹 개발

- **프런트엔드 웹 개발 시장 현황**

  현재 프런트엔드 개발의 핵심 특징은 **JavaScript 기반 프레임워크 중심**

  주요 프레임워크 점유율

  - **React**: 약 57% 이상 (압도적 1위)
  - **Vue**: 약 18%
  - **Next.js**: 약 14%
  - Angular 등 기타 프레임워크

  JavaScript/TypeScript 프레임워크 사용 비율 **약 65%**

프런트엔드 웹 개발

- **React 중심의 프레임워크 시대**

과거

JavaScript → 선택

현재

TypeScript → 기본

이유

- 대규모 프로젝트 유지보수
- 타입 안정성
- IDE 자동완성

프런트엔드 웹 개발

- **TypeScript 사실상 필수**

대표 예
- **Next.js**
- Remix
- Nuxt
- SvelteKit

특징
- SSR (Server Side Rendering)
- API 기능
- 풀스택 기능

특히 **Next.js**는 React 서버 컴포넌트 지원으로 급성장

프런트엔드 웹 개발

- **AI 기반 개발 증가**
  - AI 도구

    코드 생성

    UI 생성

    테스트 자동화

  - 사용 예

    Figma → 코드 자동 변환

    AI UI generator

    Copilot(깃허브 코파일럿) – vs code에 플러그인

    AI 기반 "**vibe coding**"도 등장

프런트엔드 웹 개발

- **프런트엔드 개발자의 실제 업무**
  - **UI 개발** : 화면구현, 반응형 디자인, 접근성 고려
  - **API 연동** : 백엔드와 통신
  - **상태관리** : 장바구니, 로그인, 사용자 정보
  - **브라우저 대응** : 모바일, 태블릿, 다양한 브라우저

- **기업에서 원하는 프런트엔드 역량**
  - **JavaScript 및 TypeScript 깊은 이해**
  - **React 또는 Vue 경험**
  - **GIT 협업**
  - **Rest API 이해**

풀스택 개발(프런트엔드 + 백엔드)

- **React + SpringBoot(Java 프레임워크)**
  - **React**

    UI 화면

    사용자 인터페이스

    API 호출

  - **Spring Boot**

    비즈니스 로직

    인증 / 권한

    데이터 처리

    DB 연결

풀스택 개발(프런트엔드 + 백엔드)

- **React + SpringBoot(Java 프레임워크)**
  - **장점**
    - Java 기반이라 **대규모 시스템에 강함**
      예) 금융 / 공공기관 / 대기업 서비스
    - 유지보수 안정성
    - 풍부한 프레임워크 생태계
  - **단점**
    - 개발 속도 느림
    - 설정 많음

풀스택 개발(프런트엔드 + 백엔드)

- **React + Nodejs(Express / NestJS)**
  - **React**

    UI 화면

    사용자 인터페이스

    API 호출

  - Express / NestJS

    비즈니스 로직

    인증 / 권한

    데이터 처리

    DB 연결

풀스택 개발(프런트엔드 + 백엔드)

- **React + Express/NestJS**
  - **장점**

    - 개발 속도 빠름

      예) 스타트 업 / 중소 기업

    - 풀스택(javascript 언어 하나로 통일)

  - **단점**

    - 대규모 시스템에서 복잡성 증가
    - 구조 관리 어려움
    - 멀티스레드 처리 약함

풀스택 개발(프런트엔드 + 백엔드)

- **아키텍쳐 비교**

항목	React + Spring Boot	React + Express
언어	Java + JavaScript	JavaScript
개발 속도	보통	빠름
안정성	매우 높음	보통
대규모 서비스	매우 적합	보통
스타트업	보통	매우 적합

HTML이란?

HTML(HyperText Markup Language)

- 하이퍼텍스트를 마크업 하는 언어, HTML5(현재 버전)
- **하이퍼텍스트** : 웹 사이트에서 링크를 클릭해 다른 문서나 사이트로 이동하는 기능
- **마크업** : **tag**(태그)를 사용해 문서에서 어느 부분이 제목이고 본문인지, 어느 부분이 사진이고 링크인지 표시하는 명령어(코드)

- **HTML**의 역사

  인물 : **팀버너스리** – 웹의 아버지, 영국의 컴퓨터 과학자

  WorldWideWeb(월드와이드웹) 하이퍼텍스트 시스템 고안 – cern(유럽 입자 물리연구소)에서 개발됨.

  URL, HTML, HTTP 최초 설계, W3C 창립

웹브라우저와 웹편집기

웹 브라우저(Web Browser)

- 웹 사이트를 둘러 볼때 사용하는 프로그램
- 크롬, 익스플로러, 파이어 폭스, 사파리 등

웹 편집기

- **비주얼 스튜디오 코드(VS code)**, 브라켓, 노트패드
- 통합개발환경(IDE) – 비주얼 스튜디오, 서브라임 텍스트

★ 최초의 웹브라우저 : 모자이크 웹브라우저

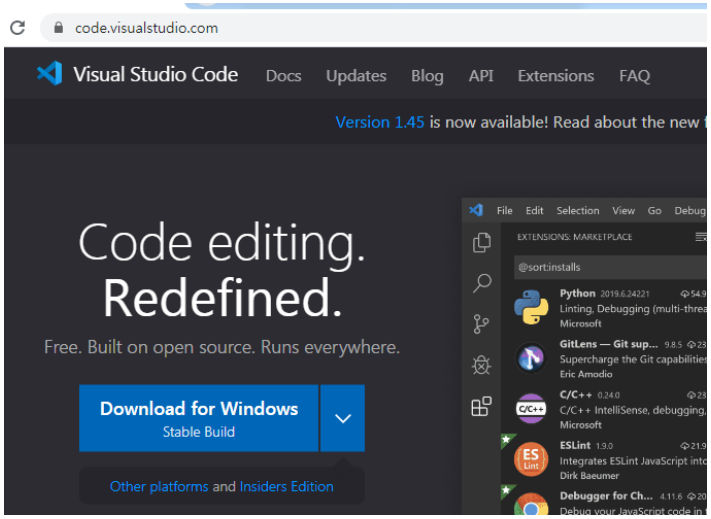
- 개발자 : 마크 안드레센(미국 대학생, 1993년 출시)

텍스트 뿐만 아니라 이미지도 표시할 수 있는 멀디미디어 GUI 브라우저 네스케이프로 발전 -> MS 익스플로러와 경쟁 후 패배 -> 모질라 파이어폭스로 계승

비주얼 스튜디오 코드 설치

◆ **비주얼 스튜디오 코드(VS code)**

- 비주얼 스튜디오 코드 다운로드 -> 설치

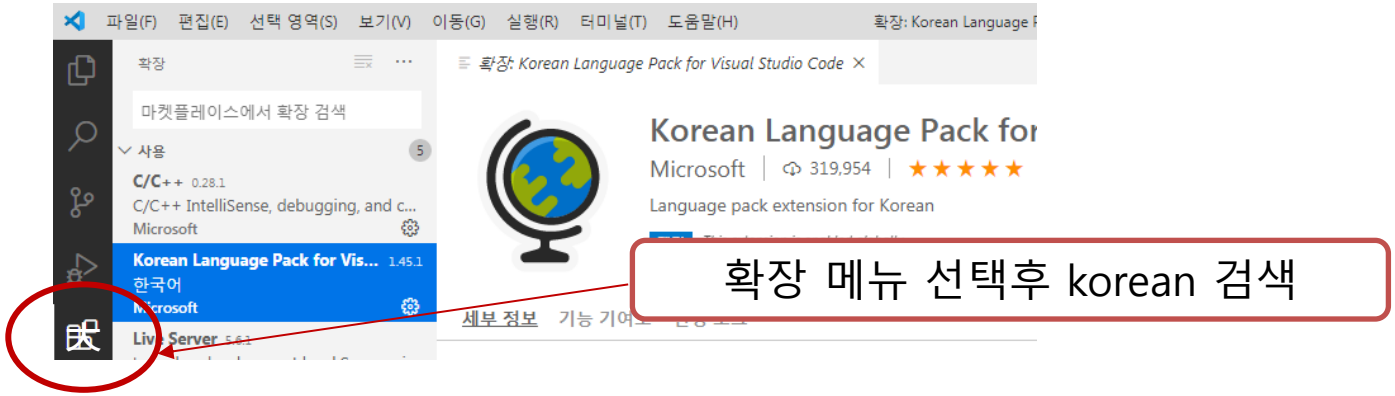


다운로드

VS 코드 화면

비주얼 스튜디오 코드 환경 설정

- 한국어 팩 설치



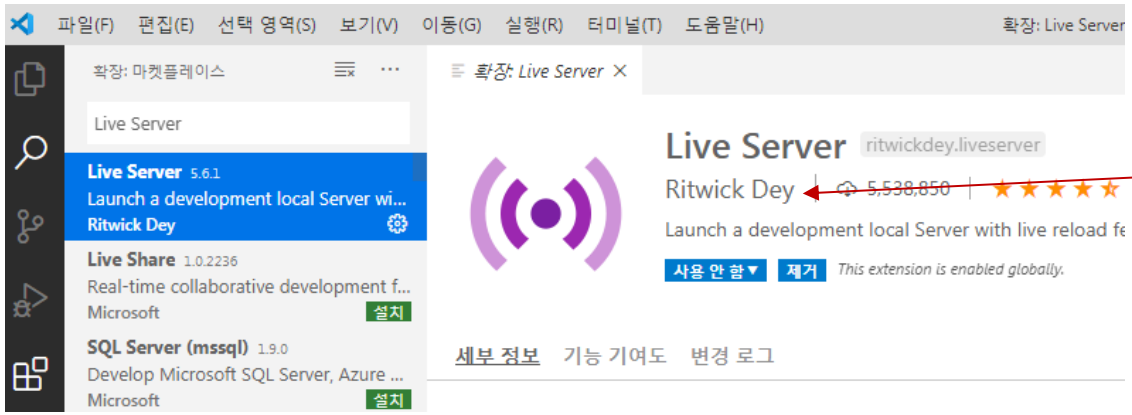
- 설정
  파일 > 기본설정 >설정

웹 서버 설치하기

◆ 웹 서버 설치

웹 서버란 HTTP를 통해 웹 브라우저에서 요청하는 HTML 문서를 보여주는(서비스 하는) 프로그램이다.

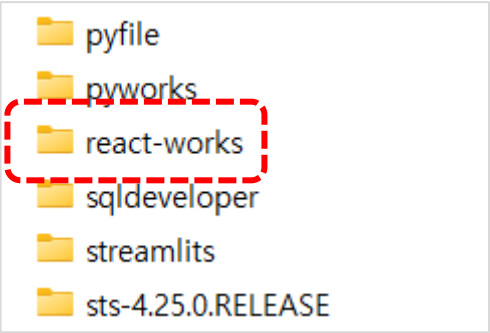
- 라이브 서버(Live Server) 설치



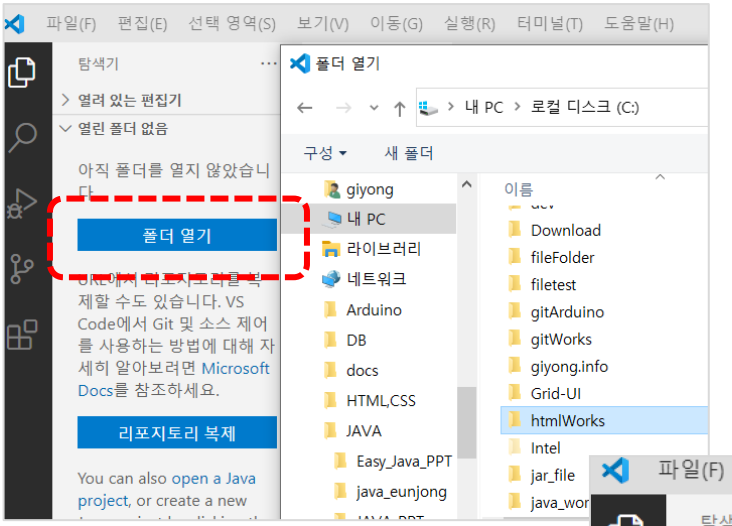
Ritwick Dey 설치

VS code – 작업 폴더 설정하기

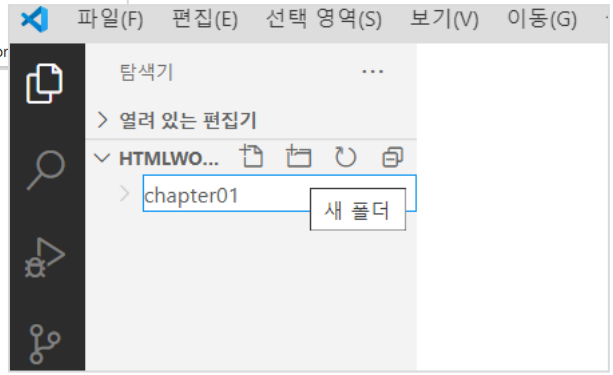
◆ **VS code – 작업 폴더 설정**



① 작업영역 폴더 만들기



② 작업영역 폴더 설정



③ 새 폴더 만들기

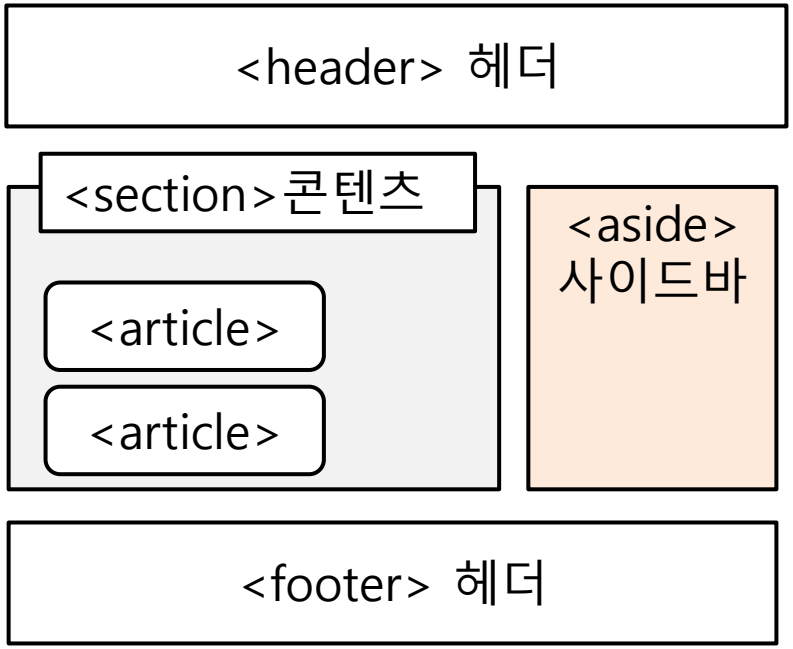
HTML5 문서

HTML5 문서

웹 문서의 표준화를 위해 레이아웃을 만들 때 사용하는 주요 태그를 미리 약속.

태그만 봐도 직관적으로 어느 부분이 헤더이고 실제 내용인지 알 수 있음.
즉, 문서구조를 정확히 이해할 수 있음.

- 2008년 1월 HTML5 초안 공개
- 2014년 11월 공식 표준 발표
- 2022년 현재 W3C에서 명세를 관리함



HTML5 문서

```
▼<header>
  ▶<div class="nav_head">…</div>
  ▼<nav>
    ▶<div class="wrap" id="gnb">…</div> scroll
  </nav>
</header>
▶<main>…</main>
<!-- 푸터 영역 -->
▼<footer>
  ▶<div class="wrap">…</div>
  ▶<div class="graybox">…</div>
```

HTML5 문서

<header> 태그 – 머리말 지정하기

- 사이트 전체의 제목 부분이 될 수도 있고, 본문의 제목 부분이 될 수도 있다.
- <nav> 태그를 사용해 사이트 메뉴를 넣을 수 있다.

<section>, <article>태그 – 주제별 컨텐츠 영역

- 문서에서 주제별로 컨텐츠를 묶을 때 사용
- 섹션 제목을 나타내는 <hn> 태그가 함께 사용됨

<aside> 태그 – 본문 이외의 내용

- 본문 내용 외에 주변에 표시되는 기타 내용들
- 광고나 링크 모음 등 문서의 메인 내용에 영향을 미치지 않는 내용

<footer> 태그 – 연락처정보, 주소

- 사이트 제작자의 연락처 정보와 저작권 정보
- 주소는 **<address>** 태그가 사용됨

HTML 태그(tag)

HTML 태그(Tag)

- 웹 문서의 내용중 '<' 와 '>'로 묶인 부분을 태그라 한다.
- 대부분의 태그는 여는 태그와 닫는 태그로 구성된다. <h2>~</h2>, ~
- 태그는 속성과 함께 사용할 수 있다. <태그 속성="속성값" 속성="속성값" ...>

태그	설명
<!DOCTYPE html>	현재 문서가 HTML5 언어로 작성된 웹 문서라는 뜻 문서 유형을 지정.
<html>~</html>	웹 문서의 시작과 끝을 나타내는 태그
<head>~</head>	웹 브라우저가 웹 문서를 해석하기 위해 필요한 정보들을 입력하는 부분. <meta>태그 – 문자 세트를 비롯한 문서 정보 <style>태그 – CSS 코드 <title>태그 – 브라우저의 제목 표시줄에 표시
<body>~</body>	실제로 웹 브라우저 화면에 나타날 내용

HTML 태그(tag)

HTML 주요 태그(Tag)

태그	설명
<hn>	Headline을 뜻하고.. <h1>제목 표시</h1> 1~6까지 있고, 1이 가장 크고 6이 가장 작은 크기 제목
	paragraph의 줄임말로 '단락'이란 앞 뒤에 줄바꿈이 있는 텍스트 덩어리를 말함. 텍스트
 	줄바꿈을 하려면 줄을 바꿀 위치에 태그를 사용한다
	divison의 약자로 html 요소를 구분하거나, 레이아웃의 틀을 구분할 때 사용하는 태그이다.
	줄바꿈없이 영역 묶기 – 일부 텍스트만 묶어 스타일을 적용하려고 할때 사용함. 내용 <

HTML 태그(tag)

HTML 주요 태그(Tag)

태그	설명
<img>	**<img src="경로" 속성="값">** **alt 속성** - 이미지를 설명하는 대체 텍스트이다. **width, height 속성** - 이미지의 크기 (너비, 높이)를 조정할 수 있다 **<img src = "images/logo.gif" width="200" height="100 alt="회사로고">**
<a>	<a>태그는 클릭했을 때 다른 페이지로 넘어가는 기능을 가지고 있다. <a href="링크할 주소" 속성="속성값">텍스트 </a> <a href="링크할 주소" 속성="속성값"><img src="이미지 파일 경로"></a>
<nav>	페이지(사이트)를 연결하는 링크들의 집합을 정의할 때 사용하는 태그이다.

간단한 HTML 문서 만들기

◆ **HTML 문서 만들기 – web.html**

127.0.0.1:5500/html/tag.html

웹 기초 기술

웹은 인터넷을 통해 정보를 주고받는 시스템
웹 페이지는 HTML, CSS, JavaScript 등의 기술로 만들어진다.

HTML

HTML은 웹 페이지의 구조를 정의하는 언어이다.
HTML 태그를 사용하여 텍스트, 이미지, 링크 등을 배치할 수 있다.

CSS

CSS는 웹 페이지의 스타일을 정의하는 언어이다.
CSS를 사용하여 글꼴, 색상, 레이아웃 등을 조절할 수 있다.

JavaScript

JavaScript는 웹 페이지에 동적인 기능을 추가하는 프로그래밍 언어이다.
JavaScript를 사용하여 사용자와 상호작용하는 기능을 구현할 수 있다.

간단한 HTML 문서 만들기

이미지

링크

네이버로 이동
청와대로 이동

간단한 HTML 문서 만들기

◆ **HTML 문서 만들기 – web.html**

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Welcome~</title>
</head>
<body>
  <h1>웹 기초 기술</h1>
  <p>웹은 인터넷을 통해 정보를 주고받는 시스템<br>
    웹 페이지는 HTML, CSS, JavaScript 등의 기술로 만들어진다.</p>
  <h2>HTML</h2>
  <p>HTML은 웹 페이지의 구조를 정의하는 언어이다. <br>
    HTML 태그를 사용하여 텍스트, 이미지, 링크 등을 배치할 수 있다.</p>
  <h2>CSS</h2>
  <p>CSS는 웹 페이지의 스타일을 정의하는 언어이다.<br>
    CSS를 사용하여 글꼴, 색상, 레이아웃 등을 조절할 수 있다.</p>
  <h2>JavaScript</h2>
  <p>JavaScript는 웹 페이지에 동적인 기능을 추가하는 프로그래밍 언어이다.<br>
    JavaScript를 사용하여 사용자와 상호작용하는 기능을 구현할 수 있다.</p>
```

간단한 HTML 문서 만들기

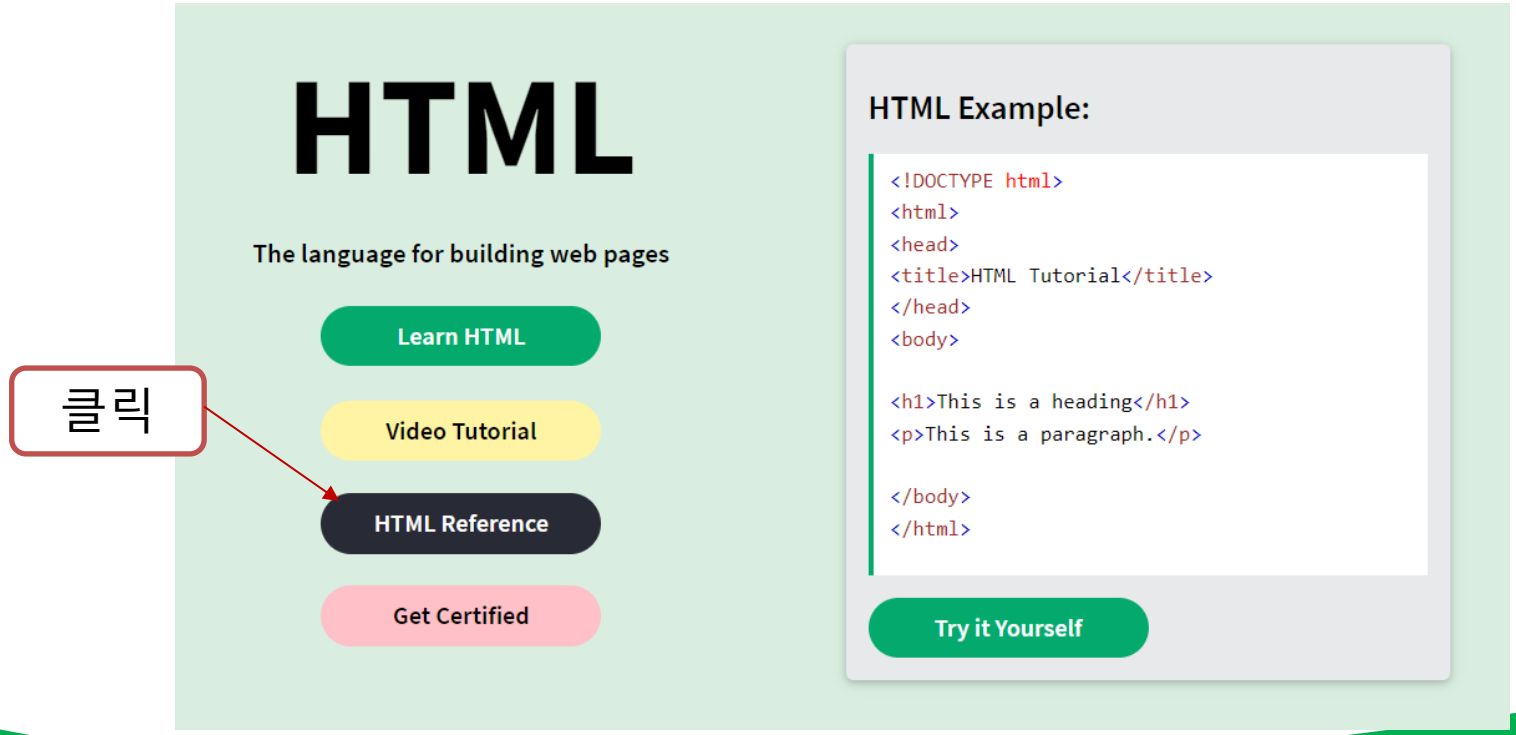
◆ **HTML 문서 만들기 – web.html**

```html
<!-- 이미지 태그 -->
<h2>이미지</h2>
<img src="spring.jpg" alt="봄 이미지" width="500"><br><br>
<img src="https://www.president.go.kr/images/202511/1763535424915_0.svg"
     alt="청와대 로고" >

<!-- 페이지 링크 -->
<h2>링크</h2>
<a href="https://www.naver.com" target="_blank">네이버로 이동</a><br>
<a href="https://www.president.go.kr" target="_blank">청와대로 이동</a><br>
<a href="https://www.president.go.kr" target="_blank"><br>
  <img src="https://www.president.go.kr/images/202511/1763535424915_0.svg"
     alt="청와대 로고" >
  </a>
</body>
</html>
```

w3schools

- **W3 스쿨즈** - https://www.w3schools.com/
  온라인으로 웹 기술을 배우는 교육용 웹 사이트이다.
  콘텐츠에는 HTML, CSS, 자바스크립트, JSON, PHP, Bootstrap, Jquery 등
  다양한 교육용 자료를 제공한다.



w3schools

<blockquote>	Defines a section that is quoted from another source
<body>	Defines the document's body
 	Defines a single line break
<button>	Defines a clickable button
<canvas>	Used to draw graphics, on the fly, via scripting (usually JavaScript)
<caption>	Defines a table caption
	Not supported in HTML5. Use CSS instead. Defines centered text

The br element

To force
line breaks
in a text,
use the br
element.

텍스트에서 강제로 줄바꿈하려면 하려면
br 태그를 사용하세요

Run »

```
<!DOCTYPE html>
<html>
<body>

<h1>The br element</h1>

<p>To force<br> line breaks<br> in a text,<br> use the br<br> element.</p>

<p>텍스트에서 강제로 줄바꿈하려면 하려면 <br>
   br 태그를 사용하세요 </p>
</body>
</html>
```

스타일과 스타일 시트

CSS(Cascading Style Sheets)란?

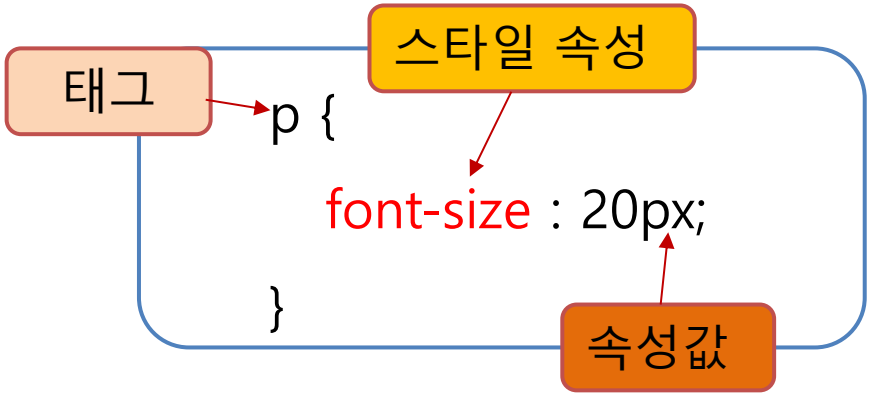
- CSS는 HTML과 함께 웹 표준 기술이다.
- **HTML**이 텍스트나 이미지, 표 같은 각 요소를 문서에 넣어 뼈대를 만드는 것이라면 **CSS**는 텍스트 색상이나 크기, 이미지 크기나 위치, 표 색상, 배치 방법 등 웹 문서의 디자인 요소를 담당한다.

스타일(Style)

- 웹에서의 스타일(style)은 미리 약속한 스타일 속성을 입력해 웹을 디자인하는 것이다.
- 웹 문서의 내용과 상관없이 디자인만 바꿀 수 있다.
- 다양한 기기에 맞게 탄력적으로 바뀌는 문서를 만들 수 있다.

스타일과 스타일 시트

스타일 형식



스타일 규칙

세미콜론(;)으로 구분하여 중괄호{ } 안에 나열한다.

스타일을 표기하는 방법

```
h2{font-size: 20px; color: blue;}

h2{
    font-size: 20px;
    color: blue;
}
```

스타일과 스타일 시트

스타일 시트의 종류

종 류	설 명
내부 스타일 시트	**반드시 <head> 태그 안에 작성한다.** **<style>** **p{font-family: 궁서체;}** **</style>**
외부 스타일 시트	<head> 태그 안에 작성한다. **<link rel="stylesheet" href="./style/style.css">**
인라인 스타일	내용 중간에 태그와 함께 사용한다. ** **블루베리는 황산화제인..

글꼴 관련 스타일

글꼴 (font) 관련 스타일

종류	설명
font-size (글자크기)	px(픽셀) 단위 – 고정된 크기, 기본 16px em단위 - 부모 요소 크기를 기준으로 한 상태 크기, p{font-size: 2em}
font-family (글자모양)	body{ font-family: 궁서 } 구글 웹폰트 : <link href="https://fonts.googleapis.com/css2?family=**Nanum+Gothic**&display>
font-weight (글자 굵기)	p{font-weight: bold}
color (글자색)	1. rgb방식 - color:rgb(0, 255, 0) -> 8bit 이해하기 2. 16진수 방식 - color:#0000ff -> 16진수 표기법 3. 기본색 방식 - color:red

문단 스타일

문단 (text) 관련 스타일

종류	설명
text-align (텍스트 정렬)	문단 정렬에는 왼쪽 / 가운데 /오른쪽 / 양쪽 정렬이 있다 text-align: left; text-align: center; text-align: right; text-align: justify;
line-height (줄 간격)	줄 사이의 간격을 조절하는 스타일이다. line-height: 2 //글자 크기의 2배
text-decoration (텍스트에 줄표시)	주로 <a> 태그의 스타일로 사용되면 줄을 표시하고 없애는 스타일이다.
text-shadow (그림자 효과)	text-shadow | <가로거리> <세로거리><번짐정도><색상>

전체/태그 선택자

선택자(Selector)란?

스타일의 속성을 적용하는 요소를 '선택자(selector)'라고 부른다. 이 선택자는 태그 하나가 될 수도 있지만 여러 개의 요소를 묶어 별도의 선택자로 지정할 수도 있다.

전체 선택자

- 스타일 모든 요소에 적용할 때 사용 - ***(별)**을 사용.
- 다른 선택자와 함께 모든 하위 요소에 한꺼번에 스타일 적용

```
* {
    margin:0;
    padding:0;
}
```

태그 선택자

- 문서에서 특정 태그를 사용한 모든 요소에 스타일 적용

```
h2 {
  color: blue; /* 글자 색 */
}
p {
  font-size:0.9em;  /* 글자 크기 */
  line-height: 3.0;  /* 줄간격 */
}
```

주요 선택자(selector)

id 선택자, class 선택자

- **id 선택자** : 문서 안에서 **한 번만 사용**한다면 id 선택자로 정의
  #(샵) 다음에 id 이름 지정
- **class 선택자** : 문서 안에서 **여러 번 반복할 스타일**이라면 클래스 선택자로 정의.
  마침표(.) 다음에 클래스 이름 지정

그룹 선택자

- 같은 스타일을 사용하는 선택자를 한꺼번에 정의
- 쉼표(,)로 구분해 여러 선택자를 나열함

```
h1{
    text-align:center;
}
h2{
    text-align:center;
}
```

→

```
h1, h2{
    text-align:center;
}
```

글꼴 관련 스타일

내부 스타일 – product.html

127.0.0.1:5500/html/product.html

세계 10대 푸드

마늘(garlic)

일해백리(一害百利)는 마늘의 별명이다. 한 가지 해가 있고 백 가지 이로움이 있다는 뜻이다.
그 한가지 해란, 아린 맛으로 인해 위에 부담을 준다는 것이다.

마늘 특유의 아린 맛은 알리신이라는 성분 때문으로 살균 및 항균 작용을 하며
마늘에 들어 있는 아연이 피로 회복에 도움을 준다.

스타일 예제

내부 스타일 – product.html

```
<h1>세계 10대 푸드</h1>
<h2>마늘(garlic)</h2>
<p class="p1">일해백리(一害百利)는 마늘의 별명이다. 한 가지 해가 있고
  백 가지 이로움이 있다는 뜻이다.<br>그 한가지 해란, 아린 맛으로 인해
  위에 부담을 준다는 것이다.</p>
<p class="p2">마늘 특유의 아린 맛은 <span style="color: red;">알리신</span>
  이라는 성분 때문으로 살균 및 항균 작용을 하며<br> 마늘에 들어 있는
  아연이 피로 회복에 도움을 준다.</p>
```

인라인 스타일

스타일 예제

내부 스타일 – product.html

```
<style>
  h1 {color: blue;}
  h2 {color: green;}
  p {
    font-family: "바탕체";
    /* 1.2em은 기본 글자 크기의 1.2배를 의미한다.
       1em 16px이므로 1.2em은 19.2px이 된다. */
    font-size: 1.2em;
  }
  .p1 {
    background-color: yellow;
    line-height: 1.5;
  }
  .p2 {
    background-color: skyblue;
    line-height: 1.5;
  }
</style>
```

스타일 예제

외부 스타일 – product2.html

```html
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>style 관련 예제</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

```css
h1 {color: blue;}
h2 {color: green;}
p {
  font-family: "바탕체";
  /* 1.2em은 기본 글자 크기의 1.2배를 의미한다.
     1em 16px이므로 1.2em은 19.2px이 된다. */
  font-size: 1.2em;
}
.p1 {
  background-color: yellow;
  line-height: 1.5;
}
.p2 {
  background-color: skyblue;
  line-height: 1.5;
}
```

style.css

목록 스타일

- **list-style-type** 속성 – list-style.html

```html
<h1>ABC 주식회사</h1>
<ul>
    <li>회사 소개</li>
    <li>제품 소개</li>
    <li>회원 가입</li>
    <li>게 시 판</li>
</ul>

<h1>123 주식회사</h1>
<ol>
    <li>회사 소개</li>
    <li>제품 소개</li>
    <li>회원 가입</li>
    <li>게 시 판</li>
</ol>
```

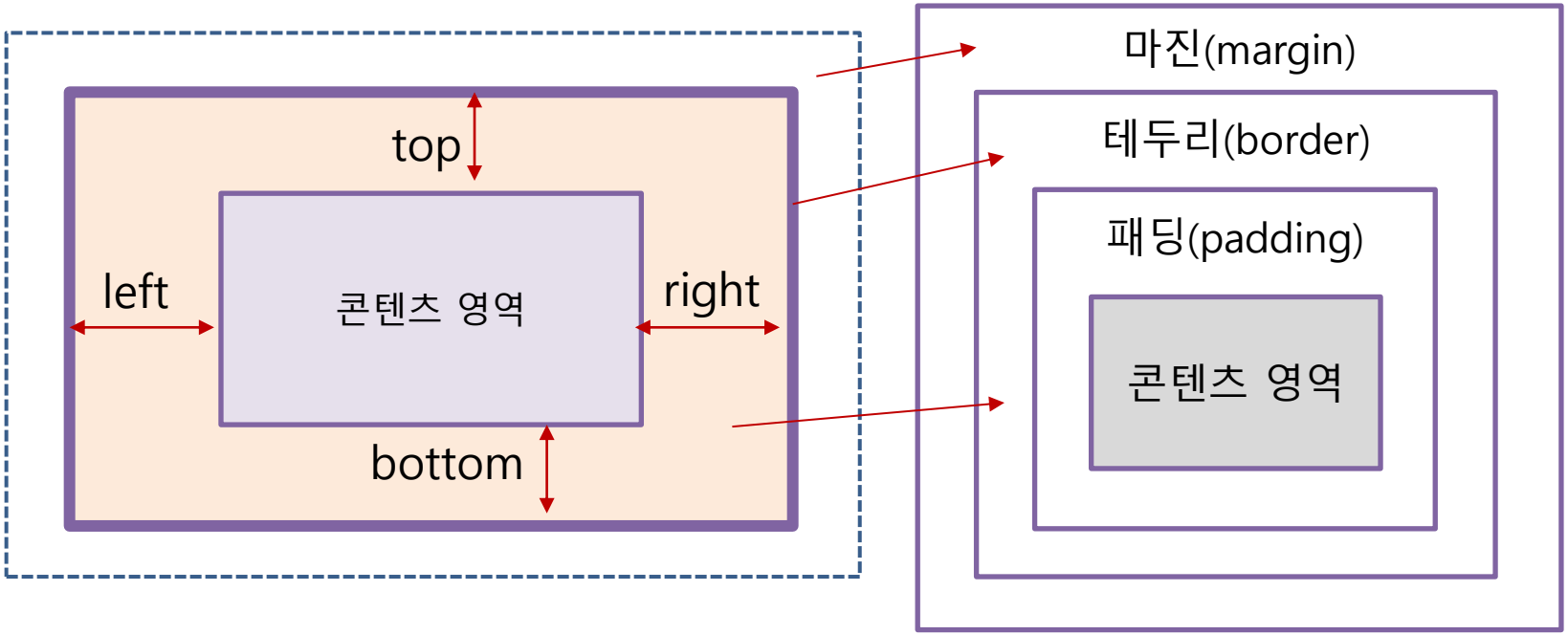
```css
ul li{
    list-style: square;
    /* list-style: none; */
}
ol li{
   /* list-style: lower-alpha; /*알파벳 소문자*/
   /*list-style: upper-roman;    /*로마자 대문자*/
   list-style: none;
}
```

CSS 박스 모델

CSS 박스모델

웹 문서에서 내용을 배치할 때는 각 요소를 박스 형태로 구성한다.

박스 모델을 실제 내용이 들어가는 콘텐츠 영역과 테두리(border), 여백(margin, padding)들로 구성된다.



바깥 여백 – margin 속성

margin 속성

- 마진(margin)은 현재 요소 주변의 여백이다.
- 한 요소와 다른 요소 사이의 간격을 조절할 수 있다.

margin: <크기> | <두께> | <색상>

margin-top: <크기> | <두께> | <색상>

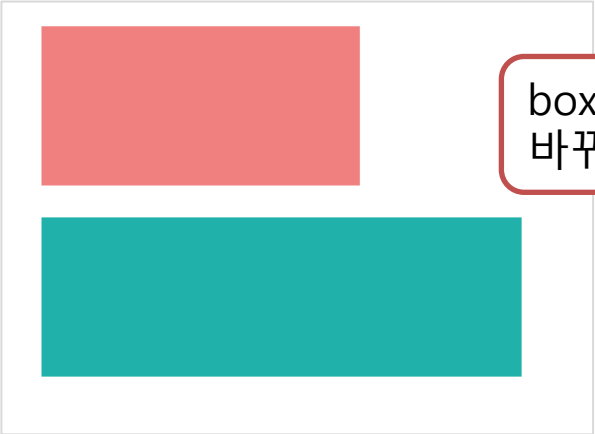
margin-right: <크기> | <두께> | <색상>

margin-bottom: <크기> | <두께> | <색상>

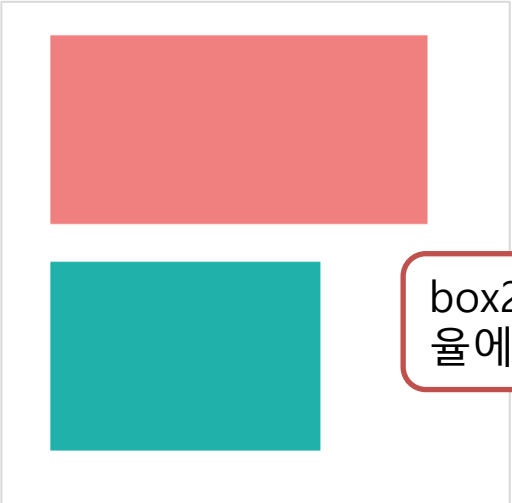
margin-left: <크기> | <두께> | <색상>

박스 모델

개체의 속성 및 여백 – box.html



box1 : 너비와 높이가 바뀌지 않음

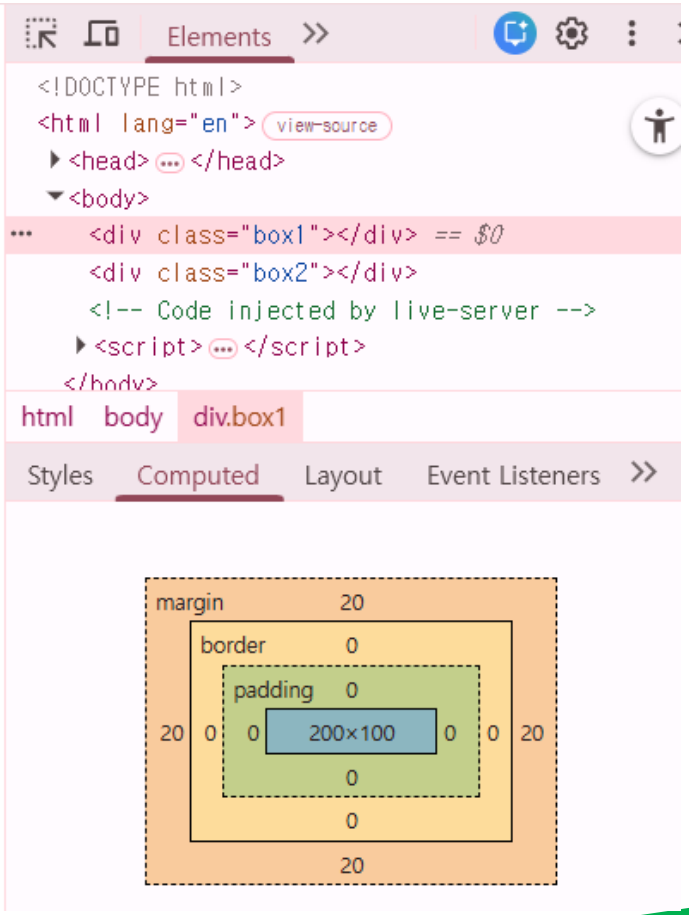


box2 : 부모 요소의 크기 비율에 따라 크기가 달라짐

박스 모델

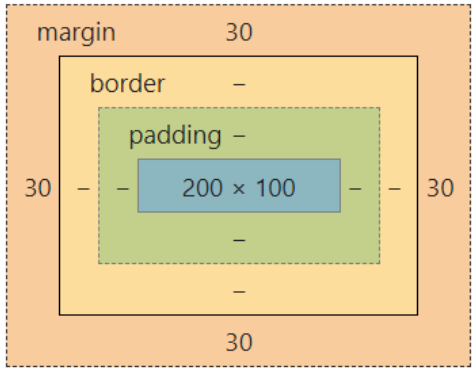
개체의 속성 및 여백 – box_model.html

```
  <style>
    .box1 {
      width: 200px;
      height: 100px;
      background-color: lightcoral;
      margin: 20px;
    }
    .box2 {
      /* width는 부모 요소의 너비에 대한 백분율로 지정 */
      width: 50%;
      height: 100px;
      background-color: lightseagreen;
      margin: 20px;
    }
  </style>
</head>
<body>
  <div class="box1"></div>
  <div class="box2"></div>
</body>
```



바깥 여백 – margin 속성

margin 속성 – 요소 주변 여백 설정하기

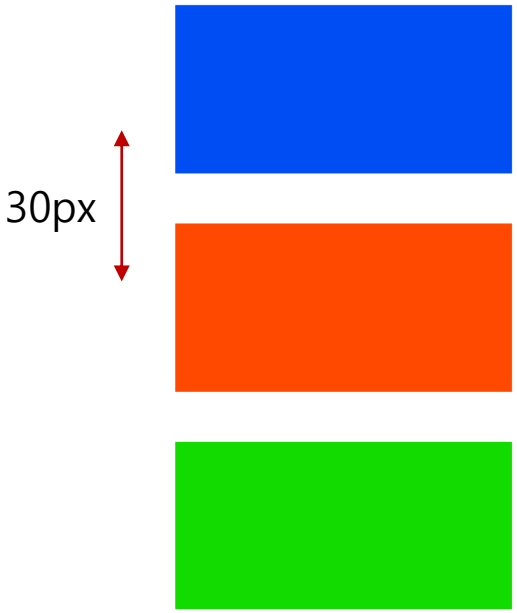



```html
<style>
  div {
    width:200px;  /* 너비 */
    height:100px;  /* 높이 */
    margin:30px;  /* 마진 */
    display:inline-block;  /* 가로로 배치 */
  }
  #box1 {
    background: rgb(0, 77, 243);
  }
  #box2 {
    background: rgb(255, 72, 0);
  }
  #box3 {
    background: rgb(18, 219, 0);
  }
</style>
```

바깥 여백 – margin 속성

margin 속성 – 요소 주변 여백 설정하기

마진 중첩 – 위 마진과 아래 마진이 서로 만날때 큰 마진값으로 합쳐지는 것을 말하는데, 가로 마진은 중첩 안됨




```
<style>
  div {
    width:200px;  /* 너비 */
    height:100px;  /* 높이 */
    margin:30px;  /* 마진 */
  }
  #box1 {
    background: rgb(0, 77, 243);
  }
  #box2 {
    background: rgb(255, 72, 0);
  }
  #box3 {
    background: rgb(18, 219, 0);
  }
</style>
```

바깥 여백 – margin 속성

margin 속성 – 가운데 정렬하기

margin-center.html

```
.box {
    width:200px;  /* 너비 */
    height:300px;  /* 높이 */
    background:#ff6a00;  /* 배경색 */
    margin:0 auto; /* 마진 - 0 auto 0 auto */
}
```

margin:0 auto

요소의 너비 값을 뺀 나머지 공간의 좌우 마진을 똑같이 맞춘다.

안쪽 여백 – padding 속성

padding 속성

- 패딩(padding)은 콘텐츠 영역과 테두리 사이의 여백을 말한다.

padding: <크기> | <두께> | <색상>

padding-top: <크기> | <두께> | <색상>

padding-right: <크기> | <두께> | <색상>

padding-bottom: <크기> | <두께> | <색상>

padding-left: <크기> | <두께> | <색상>

안쪽 여백 – padding 속성

padding 속성 – 콘텐츠 영역과 테두리 사이 여백 설정하기

패딩(padding)이란 콘텐츠 영역과 테두리 사이의 여백을 말합니다.

패딩(padding)이란 콘텐츠 영역과 테두리 사이의 여백을 말합니다.

패딩(padding)이란 콘텐츠 영역과 테두리 사이의 여백을 말합니다.

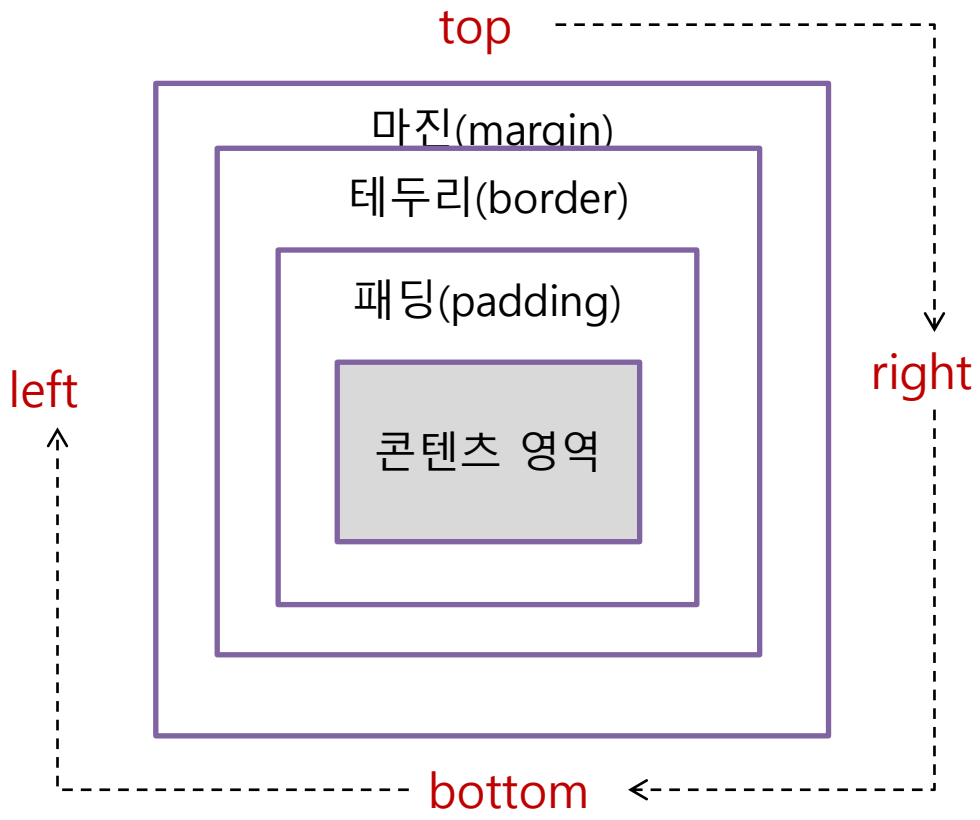
```html
<body>
    <div class="box1">패딩(padding)이란 콘텐츠영역과 테두리사이의 여백을 말합니다.</div>
    <div class="box2">패딩(padding)이란 콘텐츠영역과 테두리사이의 여백을 말합니다.</div>
    <div class="box3">패딩(padding)이란 콘텐츠영역과 테두리사이의 여백을 말합니다.</div>
</body>
```

```style
<style>
  div{
      width:200px;
      background-color: #0094ff;
      display: inline-block;
      margin: 15px;
  }
  .box1{padding:10px;}
  .box2{padding:10px 30px 10px 30px}
  .box3{padding:10px 30px}
</style>
```

테두리 관련 속성들

테두리(Border)

- 4개 방향의 값을 한꺼번에 지정할 때는 방향 순서를 지켜야 함
- border-width: top -> right -> bottom -> left



박스 모델(Box Model)

border 색상, 두께, 색상 등을 한꺼번에 묶어 표기

border: <크기> | <두께> | <색상>

border-top: <크기> | <두께> | <색상>

border-right: <크기> | <두께> | <색상>

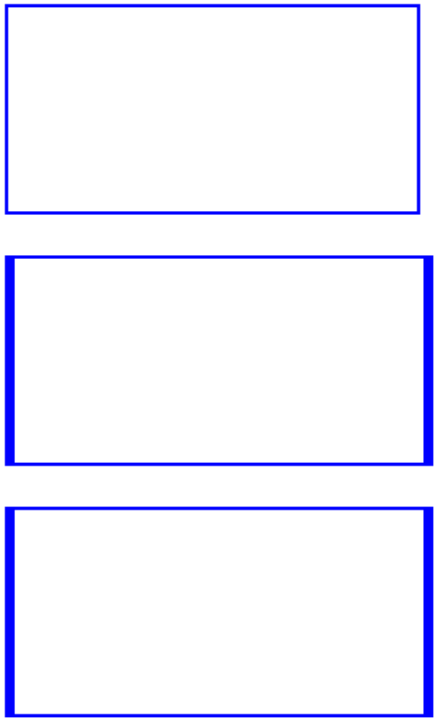
border-bottom: <크기> | <두께> | <색상>

border-left: <크기> | <두께> | <색상>

테두리 관련 속성들

border-style/ border-width / border-color

border.html

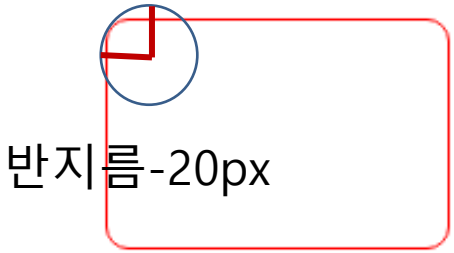


```
div{
    width: 200px;
    height: 100px;
    border-style: solid;
    border-color: blue;
    margin-bottom: 20px;
}
#box1{
    border-width: 2px;
}
#box2{
    border-width: 2px 5px;
}
#box3{
    border-width: 2px 5px 2px 5px;
}
```

테두리 관련 속성들

border-radius 속성 – 박스 모서리 둥글게 만들기

- 박스 모서리 부분을 다양한 형태로 처리할 수 있다.



```
    <style>
        img{
            border-radius: 150px;
            /*border-radius: 50%;*/
            /* px - 반지름의 크기, %는 현재 요소의 크기 기준*/
        }
    </style>
</head>
<body>
    <img src="images/photo.jpg" alt="손">
</body>
```

블록 레벨 요소와 인라인 레벨 요소

블록 레벨 요소

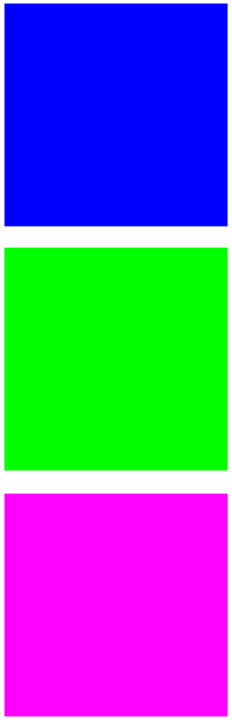
- 요소를 삽입했을 때 **혼자 한 줄**을 차지하는 요소이다.
- 요소의 너비가 100%라는 의미
- ** , <p>** 태그 등

인라인 레벨 요소

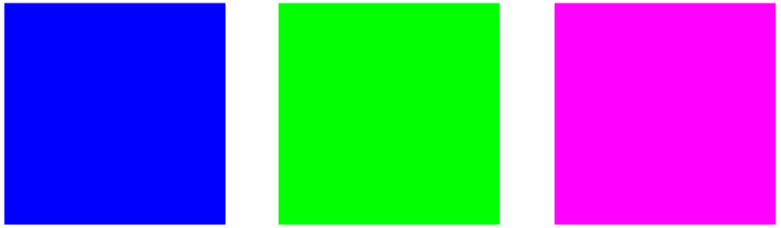
- 줄을 차지하지 않는 요소
- 화면에 표시되는 콘텐츠 만큼만 영역을 차지하고 나머지 공간에는 다른 요소가 들어올 수 있다.
- **, <img>** 태그 등

display 속성

display 속성



display: block



display: inline-block

display 속성

display 속성

```css
div{
    width: 100px;
    height: 100px;
    margin: 10px;
    /* display: block; */
    display: inline-block;
}
.box1{background: #00f;}
.box2{background: #0f0;}
.box3{background: #f0f;}
```

display.html

```html
<body>
  <div>
    <div class="box1"></div>
    <div class="box2"></div>
    <div class="box3"></div>
  </div>
</body>
```

display:inline-block

- 상단 메뉴 만들기

display:inline-block

- 상단 메뉴 만들기

```
  <title>ABC 주식회사</title>
  <link rel="stylesheet" href="css/navbar.css">
</head>
<body>
  <header>
    <nav>
      <ul>
        <li><a href="#">회사 소개</a></li>
        <li><a href="#">제품 소개</a></li>
        <li><a href="#">고객 지원</a></li>
        <li><a href="#">문의하기</a></li>
      </ul>
    </nav>
  </header>

  <section>
    <h1>ABC 주식회사</h1>
  </section>
</body>
```

navbar1.html

display:inline-block

- 상단 메뉴 만들기

navbar.css

```
body{margin: 0;}
nav ul{
  list-style: none;
  margin: 0;
  padding: 0;
  text-align: center;
}
nav ul li{
  display: inline-block;
  margin: 16px;
  border-top: 3px solid #222;
  padding: 10px 0;
}
nav ul li a{
  text-decoration: none;
  color: #333;
  padding: 10px;
}
nav ul li a:hover{background-color: #eee;}
section h1{padding: 20px;}
```

그리드 레이아웃

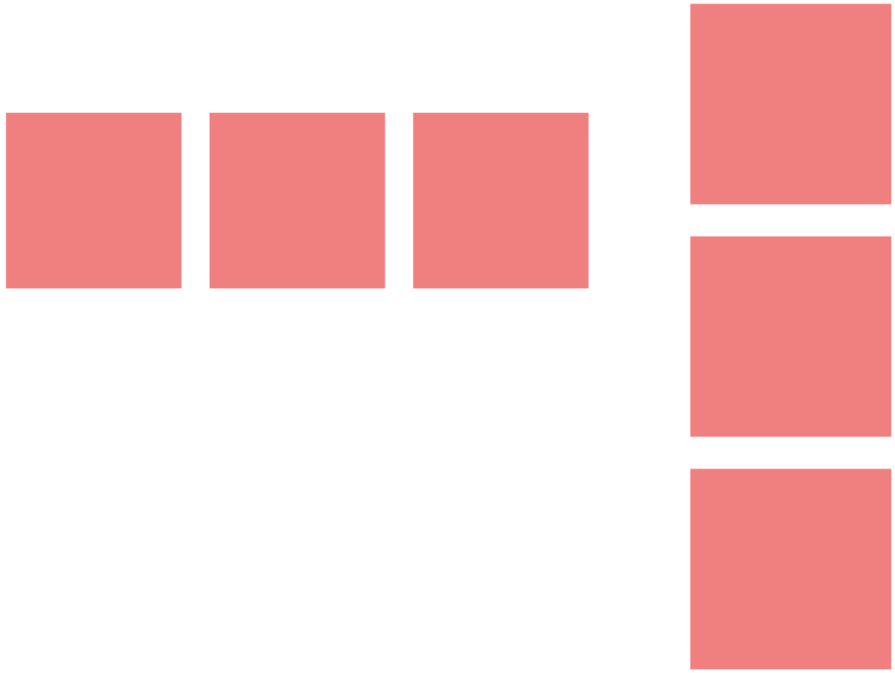
- **그리드 레이아웃(Grid Layout)**
  - 반응형 웹 디자인에서 웹 문서 요소를 배치하는 기준
  - 웹 사이트 화면을 여러 개의 칼럼(column)으로 나눈 후 웹 요소를 배치
  - 화면을 규칙적으로 배열하므로 레이아웃을 일관성 있게 유지

- **플렉스 박스(Flex Box) 레이아웃**
  - 수평이나 수직 방향 중 하나를 주축으로 정하고 박스를 배치
  - 여유 공간이 생길 경우 너비나 높이를 적절하게 늘리거나 줄일 수 있음

flex container

레이아웃 – flex

- flex-direction : 가로(row), 세로(column) 방향 배치



레이아웃 – flex

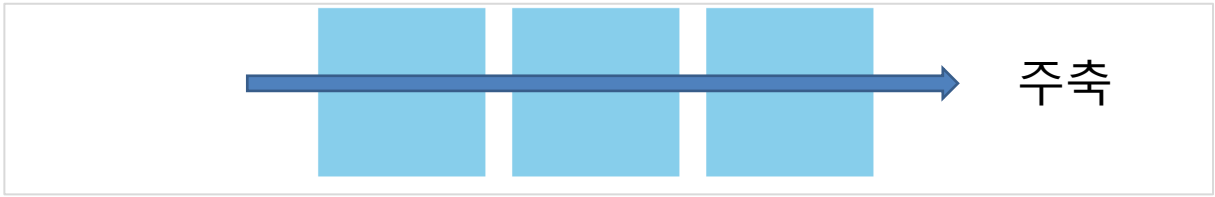
- **flex-direction : 가로(row), 세로(column) 방향 배치**

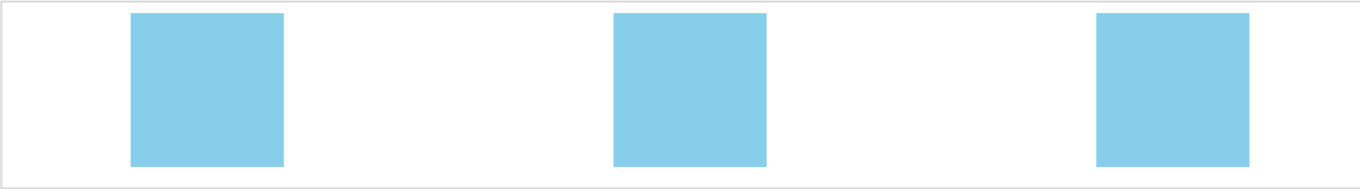
```
<style>
  .container {
    display: flex;
    flex-direction: row; /* 아이템을 가로 방향으로 배치 */
    flex-direction: column; /* 아이템을 세로 방향으로 배치 */
    gap: 16px; /* 아이템 사이의 간격 */
  }
  .box {
    width: 100px;
    height: 100px;
    background-color: lightcoral;
  }
</style>
```

```
<div class="container">
  <div class="box"></div>
  <div class="box"></div>
  <div class="box"></div>
</div>
```

레이아웃 – flex

- justify-content : 주축(main axis)의 정렬 방법을 지정





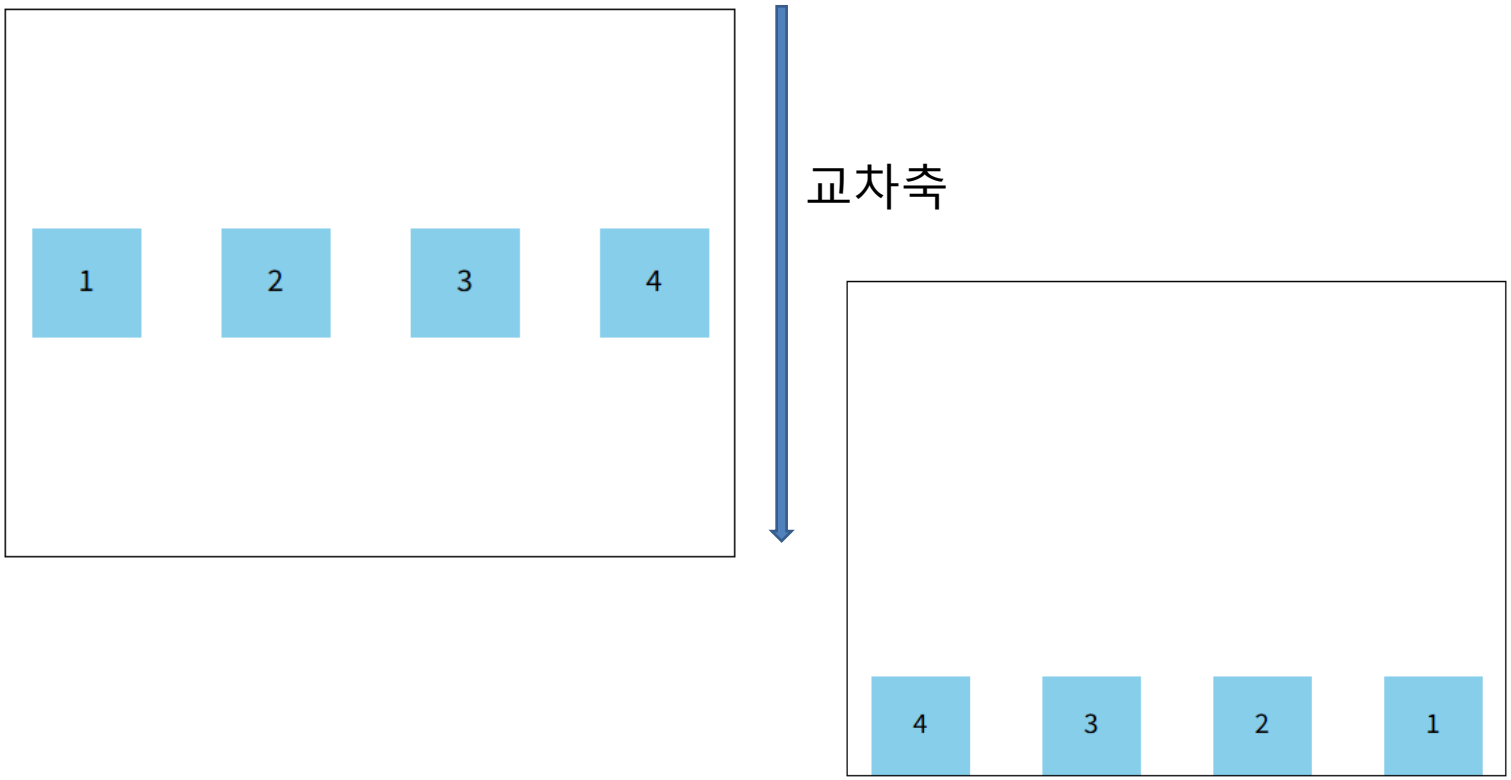
레이아웃 – flex

- **justify-content : 주축(main-axis)의 정렬 방법을 지정**

```
<style>
  .container {
    display: flex;
    justify-content: center;  /* 아이템을 가로 방향으로 중앙 정렬 */
    justify-content: flex-start;  /* 아이템을 가로 방향으로 시작 위치에 정렬 */
    justify-content: space-between;  /* 아이템을 가로 방향으로 양 끝에 정렬 */
    justify-content: space-around; /* 아이템을 가로 방향으로 고르게 분배 */
    gap: 16px; /* 아이템 사이의 간격 */
  }
  .box {
    width: 100px;
    height: 100px;
    background-color: skyblue;
  }
</style>
```

레이아웃 – flex

- **align-items : 교차축(cross axis)의 정렬 방법을 지정**



레이아웃 – flex

- **align-items : 교차축(cross-axis)의 정렬 방법을 지정**

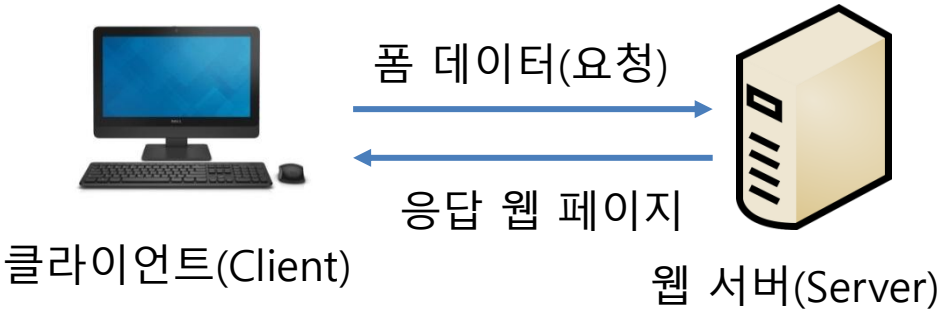
```css
.container {
  width: 400px;
  height: 300px;
  border: 1px solid #000;
  display: flex;
  align-items: center; /* 아이템을 세로 방향으로 중앙 정렬 */
  align-items: start; /* 아이템을 세로 방향으로 시작 위치에 정렬 */
  align-items: end; /* 아이템을 세로 방향으로 끝 위치에 정렬 */
  justify-content: space-around; /* 아이템을 가로 방향으로 고르게 분배 */
  justify-content: flex-start;  /* 아이템을 가로 방향으로 시작 위치에 정렬 */
  flex-direction: row-reverse; /* 아이템을 가로 방향으로 역순 배치 */
  gap: 16px; /* 아이템 사이의 간격 */
}
.box {
  width: 60px;
  height: 60px;
  background-color: skyblue;
}
p{text-align: center;}
```

```html
<div class="container">
  <div class="box"><p>1</p></div>
  <div class="box"><p>2</p></div>
  <div class="box"><p>3</p></div>
  <div class="box"><p>4</p></div>
</div>
```

폼(Form) 만들기

웹에서 만나는 폼(form)

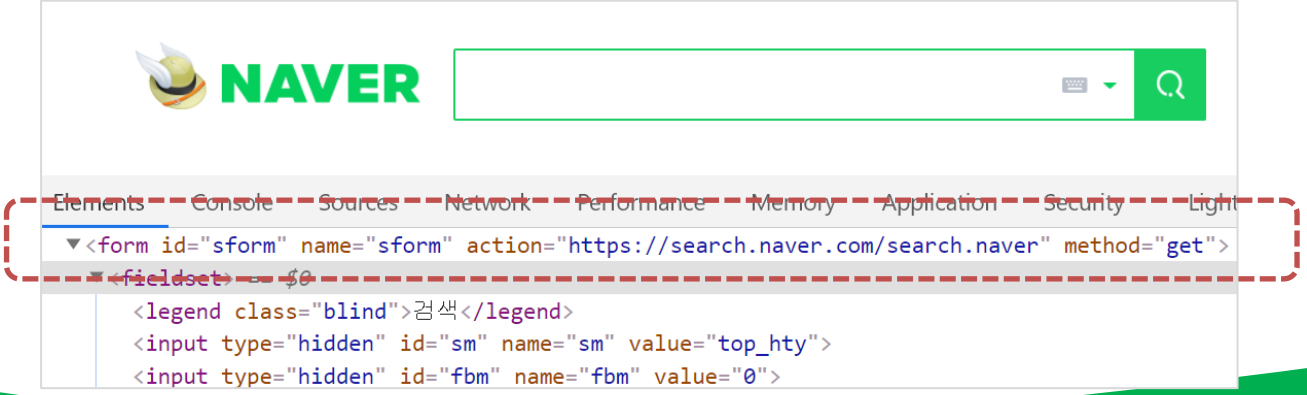
- 폼은 사용자가 웹 브라우저를 통해 입력한 데이터를 웹 서버로 전송하는 양식이다.
- 폼에 입력한 사용자 정보를 처리하는 것은 Java(JSP), Python, PHP과 같은 서버 측 웹 프로그래밍 언어이다.



폼 만들기

- **<form>태그**
  - <form 속성="속성 값">여러 폼 요소 </form>
  - 몇 가지 속성을 통해 사용자가 입력한 자료들을 서버 쪽으로 넘기는데 이용

속성	설명
action	<form> 태그 안의 내용들을 처리해 줄 서버 상의 경로를 지정한다.
method	**get** – 주소 표시줄에 사용자가 입력한 내용이 그대로 드러난다.
	post – 사용자가 입력한 내용이 드러나지 않는다.(보안이 필요한 경우)
name	폼의 이름을 지정합니다.



사용자 입력을 위한 〈input〉 태그

- **<input> 태그**

  type 속성 값에 따라 함께 사용할 수 있는 속성들도 달라진다.

유형	설명
text	한 줄짜리 텍스트를 입력할 수 있는 텍스트 상자를 넣는다.
password	비밀번호를 입력할 수 있는 필드를 넣는다.
submit	사용자 입력 내용을 서버로 전송
reset	사용자 입력 내용 전부 삭제
number	숫자 입력 필드(value, min, max, step 속성)
date	날짜를 선택할 수 있도록 표시
radio	여러 항목 중 하나만 선택 가능
checkbox	여러 항목 중 다중 선택 가능함
file	파일을 첨부할때 사용

폼 만들기

● 검색 폼 만들기

검색

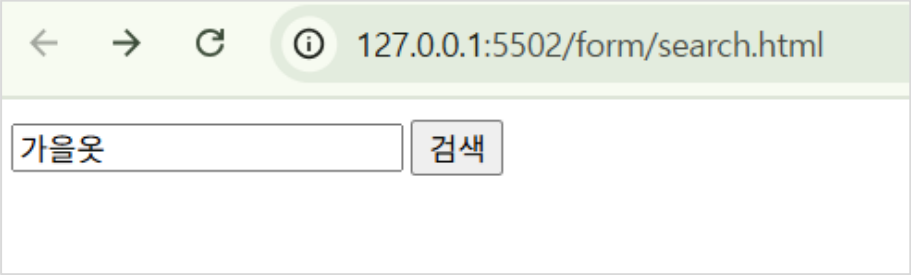
search.html

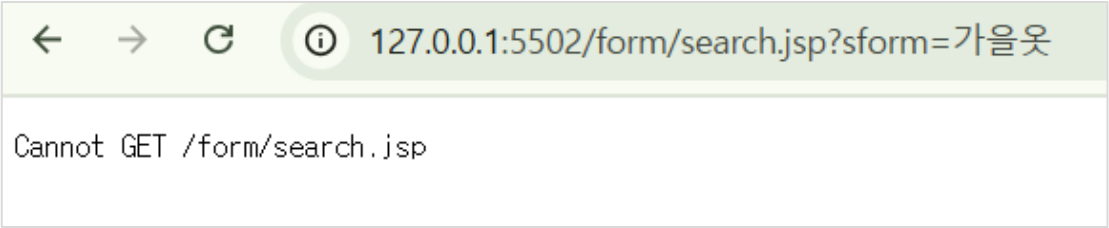
```
<!-- action = 이동경로(서버페이지), method=보내는 방법 -->
<!-- method: get(주소표시줄에 노출), post(주소표시줄에 노출안됨) -->
<!-- submit(폼을 서버에 전송), text: 입력상자 -->
<form action="search.jsp" method="get">
    <input type="text" name="sform">
    <input type="submit" value="검색">
</form>
```

<form>태그에서 action="search.jsp"로 지정했기 때문에 내용을 입력하고 검색버튼을 누르면 그 값이 웹 서버에 있는 search.jsp 파일로 전송되어 검색이 처리된다.

폼 만들기

- **검색 폼 만들기**

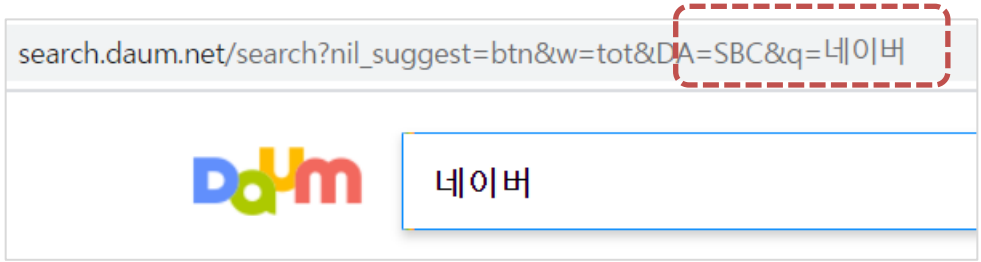

127.0.0.1:5502/form/search.html
가을옷
검색


127.0.0.1:5502/form/search.jsp?sform=가을옷
Cannot GET /form/search.jsp

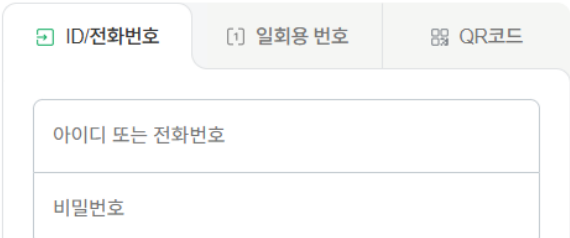
폼 만들기

- **method - get 과 post**

get 방식 – 주소표시줄에 입력 내용이 표시됨



post 방식 – 주소표시줄에 입력한 내용이 표시되지 않음

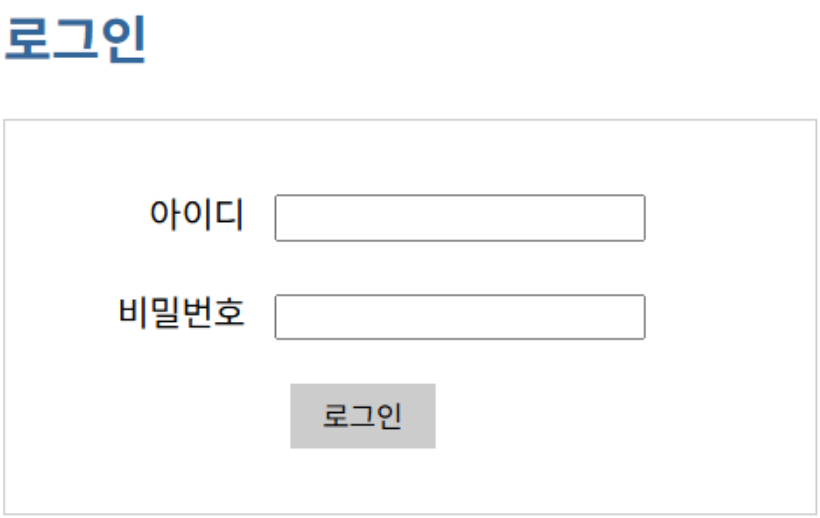


아이디, 비밀번호를 입력하고 로그인을 요청하면 주소표시줄에 나타나지 않음

```
▼<form id="frmNIDLogin" name="frmNIDLogin" target="_top" autocomplete="off" action="https://nid.naver.com/nidlogin.login" method="POST"> == $0
  <input type="hidden" id="localechange" name="localechange" value>
  <input type="hidden" name="dynamicKey" id="dynamicKey" value="XO-vDYZgA4-W29mOJ15OH_aPol__mQdteZgtVWo4ofujC5tF1TMwKhM7-MulLtKhN9f8PRFrCuCZD48124B4h9mv47o4tkH8R9Ar5LxUns">
  <input type="hidden" name="eccpw" id="eccpw" value>
```

〈label〉 태그

- **<label>** 태그
  - 입력 창 옆에 '아이디 ' 나 '비밀번호 ' 처럼 붙여 놓은 텍스트
  - 스마트 폰 처럼 화면이 작은 기기에서 라디오 버튼이나 체크박스 부분을 정확히 터치하는 것이 어려울 때 사용하면 더욱 유용하다.



〈label〉 태그

- **<label> 태그**

```
<h2>로그인</h2>
<form action="login.jsp" method="post">
    <ul>
        <li>
            <label for="userid">아이디</label>
            <input type="text" name="userid" id="userid">
        </li>
        <li>
            <label for="userpw">비밀번호</label>
            <input type="password" name="userpw" id="userpw">
        </li>
        <li>
            <button type="submit">로그인</button>
        </li>
    </ul>
</form>
```

〈label〉 태그

- **<label> 태그**

```
h2{
    margin: 20px;
    color: #369;
}
form{
    width: 350px;
    border: 1px solid #ccc;
    padding: 10px;
    margin: 20px;
}
form ul{
    padding: 0;
    margin: 0;
}
ul li{
    list-style: none;
    margin: 20px;
}
```

```
label{
    display: inline-block;
    width: 80px;
    text-align: right;
    margin-right: 10px;
}
button{
    margin-left: 100px;
    padding: 5px 15px;
    background: #ccc;
    border: none;
    cursor: pointer;
}
```

사용자 입력을 위한 〈input〉 태그

- **radio버튼 & checkbox 버튼**
  - radio 버튼 – 1개의 항목만 선택 가능, name 속성의 값이 같아야 함
  - checkbox – 여러 개의 항목 선택 가능

사용자 입력을 위한 〈input〉 태그

- **<fieldSet>, <legend> 태그 – 폼 요소 그룹으로 묶기**

  <fieldSet>태그는 폼의 요소들을 하나의 영역으로 묶어 외곽선을 그려주고, <legend>태그는 <fieldSet>태그로 묶은 그룹에 제목을 붙여 준다.

radio-check.html

```
<form action="" method="post">
    <!-- <fieldset>은 폼의 요소를 하나의 영역으로 묶어서 외곽선을 그려줌
        <legend>은 묶은 그룹에 제목을 붙여준다.
        <label>로 감싸면 글자를 선택해도 적용됨 -->
    <fieldset>
        <legend>신청 과목</legend>
        <p>이 달에 신청할 과목을 선택하세요(1과목만 가능함)</p>
        <!-- name의 subject는 동일하게 작성 -->
        <label><input type="radio" name="subject" value="speaking">회화</label>
        <label><input type="radio" name="subject" value="grammer">문법</label>
        <label><input type="radio" name="subject" value="writing">작문</label>
    </fieldset>
    <fieldset>
        <legend>메일링</legend>
        <p>메일로 받고 싶은 뉴스 주제를 선택해 주세요(복수 선택 가능함)</p>
        <label><input type="checkbox" name="mailing" value="news">해외단신</label>
        <label><input type="checkbox" name="mailing" value="dialog">5분 회화</label>
        <label><input type="checkbox" name="mailing" value="pops">모닝팝스</label>
    </fieldset>
</form>
```

사용자 입력을 위한 〈input〉 태그

- **<fieldSet>, <legend>** 태그 – **폼 요소 그룹으로 묶기**

```
form{
    width: 500px;
    border: 1px solid #ccc;
    padding: 10px;
}
fieldset{
    margin-bottom: 16px;
    padding: 10px;
    border: 1px solid #ccc; /* 외곽선 */
}
legend{
    font-weight: bold;
    padding: 0 5px; /* 좌우 여백 */
}
```

여러 줄 입력하는 텍스트 영역

- **<textarea> 태그 – 여러 줄 입력하는 텍스트 영역 만들기**
  - 폼에서 여러 줄의 텍스트를 입력할 때 사용하는 폼이다.
  - cols 속성 – 몇 글자를 쓸지 지정
  - rows 속성 - 화면에 몇 줄을 표시할 지 지정
  - Placeholder 속성 – 힌트 안내문

배송 업무

배송시 요청사항을 적어주세요

전송

여러 줄 입력하는 텍스트 영역

- **<textarea> 태그 – 여러 줄 입력하는 텍스트 영역 만들기**

textarea.html

```html
<!-- 여러줄 입력하는 기능, 멀티라인 태그
     cols(칼럼) 영어기준 글자수, rows-보이는 줄수
     placeholder-힌트글 -->
<form action="" method="post">
    <fieldset>
        <legend>배송 업무</legend>
        <textarea cols="60" rows="5" name="memo"
                  placeholder="배송시 요청사항을 적어주세요"></textarea>
    </fieldset>
    <input type="submit" value="전송">
</form>
```

```css
form{
    width: 400px;
}
textarea{
    /* 크기 조정 불가 */
    resize: none;
}
/* input 태그의 속성에 스타일을 줄때 [속성='속성값']를 사용함 */
input[type='submit']{
    margin-top: 10px;
    padding: 5px 10px;
}
```

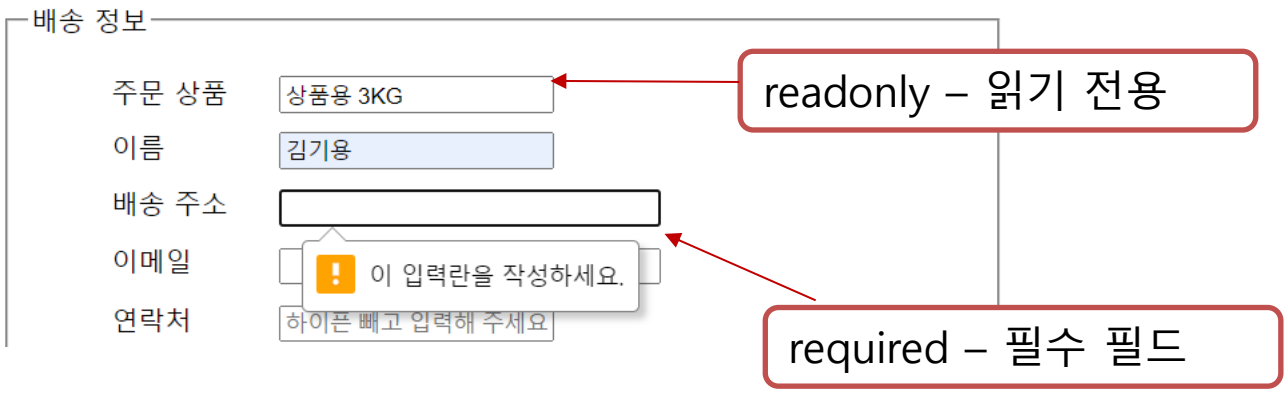
〈input〉 태그의 다양한 속성

readonly 속성 – 읽기 전용 필드 만들기

- 사용자가 입력하지 못하고 읽게만 만들 수도 있다.

required 속성 – 필수 필드 만들기

- 내용을 폼에 입력한 후 submit 버튼을 클릭하면 폼을 서버로 전송하는데 이때 필수 필드에 필요한 내용이 모두 채워졌는지 검사해야 한다.
- 필수적으로 입력해야 하는 내용에는 required 속성을 지정해 필수 필드로 만든다.



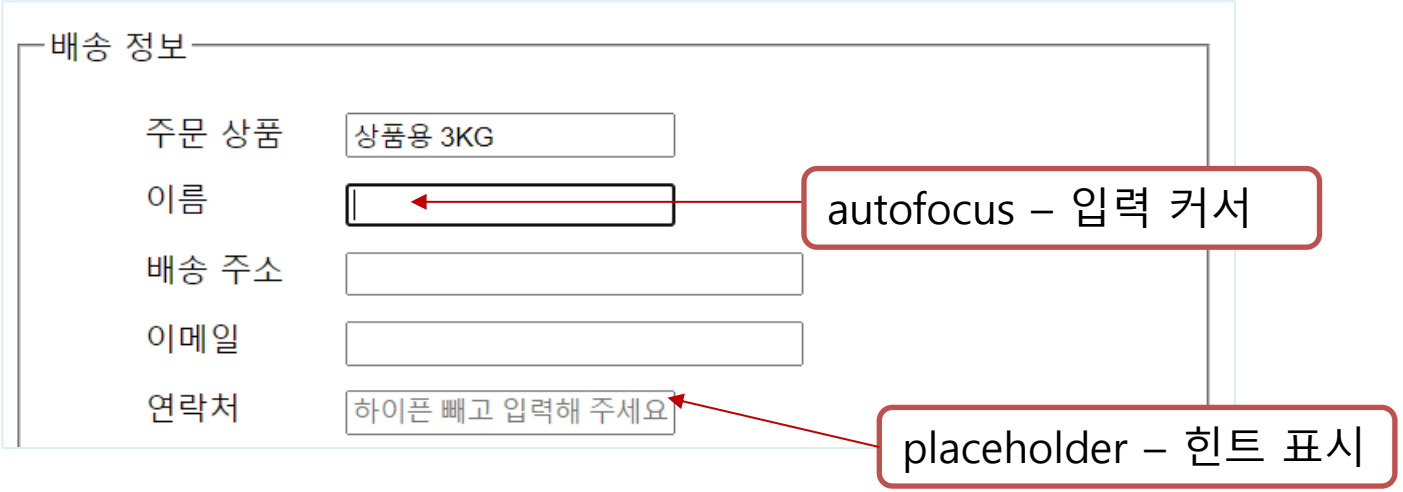
〈input〉 태그의 다양한 속성

autofocus 속성 – 입력 커서 표시하기

- 폼의 요소 중에서 원하는 곳에 마우스 커서를 표시할 수 있다.

placeholder 속성 – 힌트 표시하기

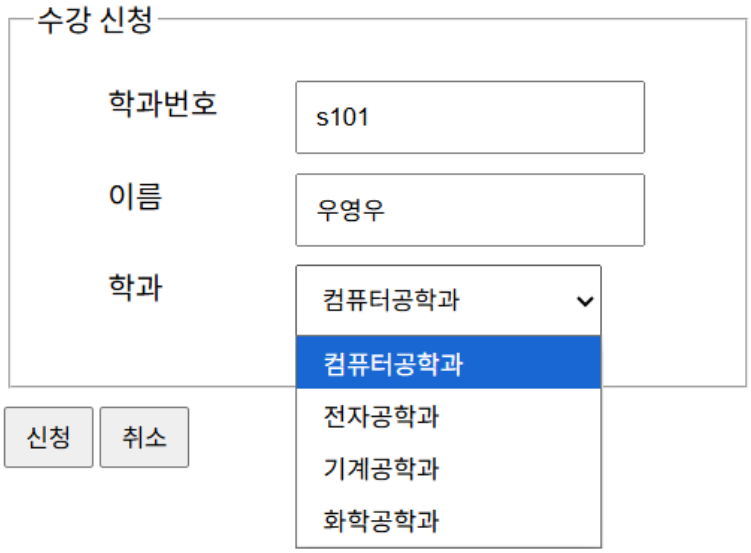
- 사용자가 텍스트를 입력할 때 도움이 되도록 입력란에는 적당한 힌트 내용을 표시하고 있다가 그 필드를 클릭하면 내용이 사라지도록 만들 수 있다.



여러 데이터 나열해 표시하기

- **<select> ~<option> 태그**
  - 목록(리스트) 형태로 여러 데이터를 나열해 표시하고 선택하는 태그

```
<select>
  <option value="c1">내용1</option>
  <option value="c2">내용2</option>
  <option value="c3">내용3</option>
</select>
```



여러 데이터 나열해 표시하기

subject.html

```
<div id="wrap">
    <form action="process.jsp" method="post">
        <fieldset>
            <legend>수강 신청</legend>
            <!-- value: 입력상자에 표시될 값
                 readonly: 읽기 전용 -->
            <ul>
                <li>
                    <label for="cid">학과번호</label>
                    <input type="text" id="cid" name="cid"
                           autofocus required>
                </li>
                <li>
                    <label for="sname">이름</label>
                    <input type="text" id="sname" name="sname"
                           value="홍길동" readonly>
                </li>
```

여러 데이터 나열해 표시하기

```
                <li>
                    <label for="major">학과</label>
                    <select id="major" name="major">
                        <option value="software">소프트웨어학과</option>
                        <option value="mechanic">기계공학과</option>
                        <option value="electric">전기전자공학과</option>
                        <option value="computer">컴퓨터공학과</option>
                    </select>
                </li>
            </ul>
        </fieldset>
        <div>
            <input type="submit" value="신청하기">
            <input type="reset" value="취소하기">
        </div>
    </form>
</div>
```

여러 데이터 나열해 표시하기

```
#wrap{
  width: 400px;
  margin: 0 auto;
}
form{
  display: flex;
  flex-direction: column; /* 세로 방향으로 배치 */
  gap: 10px; /* 요소 간의 간격 */
}
ul li{
  list-style: none;
  margin: 10px 0;
}
ul li label{
  width: 100px;
  float: left;  /* 라벨을 왼쪽으로 정렬 */
}
```

major.css

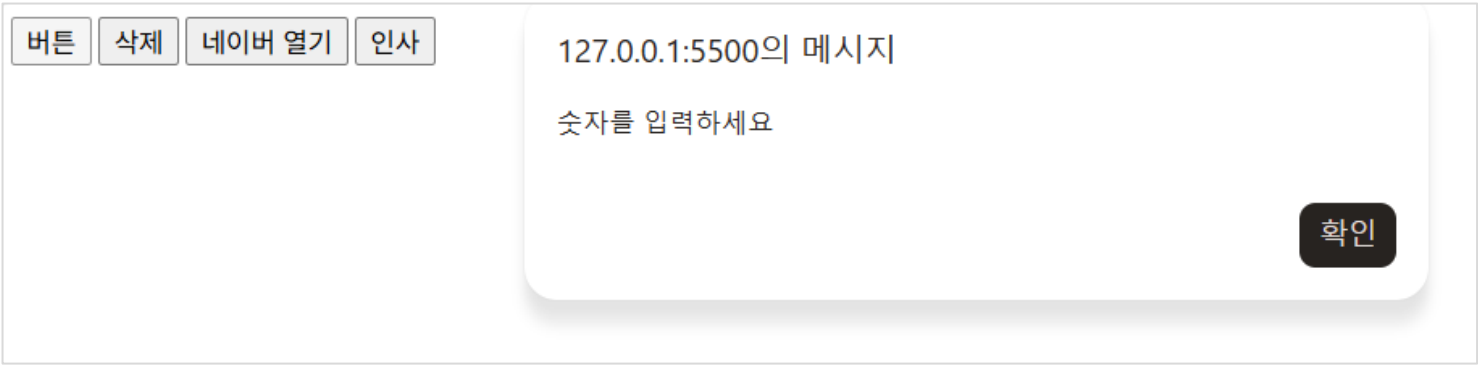
여러 데이터 나열해 표시하기

```
ul li input{
  width: 50%;
  padding: 10px;
}
select{
  width: 50%;
  padding: 10px;
}
input[type="submit"], input[type="reset"]{
  padding: 5px 10px;
  cursor: pointer;
}
```

버튼 〈button〉 태그

- **<button>태그 – 버튼 만들기**

<button>이름</button> : submit 처럼 서버 페이지로의 전송기능은 없고, **onclick** 속성을 사용하여 주로 자바스크립트 함수와 함께 사용한다.
(단, type 속성에 submit을 명시하면 서버 페이지 전송이 가능함)



버튼 〈button〉 태그

- **<button>태그 – 버튼 만들기**

```
<div>
  <button type="button" onclick="alert('숫자를 입력하세요')">버튼</button>
  <button onclick="confirm('정말로 삭제하시겠어요?')">삭제</button>
  <button onclick="window.open('https://www.naver.com', '_blank')">네이버 열기</button>
  <button onclick="greet()">인사</button>
```

버튼 〈button〉 태그

- **<button>태그 – 버튼 만들기**

```
<script>
  /*function greet() {
    const name = prompt('이름을 입력하세요');
    if (name) {
      alert(`안녕하세요, ${name}님!`);
    } else {
      alert('이름을 입력하지 않으셨네요.');
    }
  }*/

  const greet = () => {
    const name = prompt('이름을 입력하세요');
    if (name) {
      alert(`안녕하세요, ${name}님!`);
    } else {
      alert('이름을 입력하지 않으셨네요.');
    }
  }
</script>
```